

1st Assignment

Course Site @GitHub

Student: Navarrete Tepozán Iván

Due Date: Wed 4 February

Warm-up Task

Problem

Read the Written Work Guidelines and documents on mathematical writings referenced therein. Write a short paragraph (≈ 5 lines) reflecting on one major error present in your math writing hitherto, and outlining some steps you will take to improve your writing for this course.

Reflection. There is one major mistake that I have been making over the years: sometimes I refuse to write a more elaborate explanation of the problem, because I assume that anyone reading my demonstration must have an equivalent level of mathematical knowledge. However, after reviewing the resource that was shared, I realize this is a mistake. A proof should be written so that it stands on its own, without relying on the reader to fill in the gaps. $\square$

Task I

Open the Jupyter notebook [intro_feedforward_mnist.ipynb](#) which creates and trains a simple feedforward-network to learn to recognize handwritten digits in the MNIST "database". Make sure to look at the MNIST Wikipedia page and Figure 1.9 in Deep Learning Ch. 1.

Problem Sub-task 1

Play around assigning different values to the parameters, (`dim_hl_1`, `dim_hl_2`) and the `num_epochs`. Make a table and provide a human-readable analysis of your findings.

Problem For extra credit

For extra credit +20%, also compare the 2-hidden layer feedforward NN to NNs with 1 or 3 hidden layers.

Solution. It is clear that the more layers and epochs there are, the more accurate the model will tend to be. However, according to this table, we can see that it is not necessary to build a super complex model. While it is true that an 18×128 hidden layer model achieves a much higher accuracy than the other cases, the reality is that we can achieve an accuracy almost as good with much lighter models. Take the 64×64 model, for example: it uses significantly fewer parameters yet delivers performance comparable to the more complex model. The main point is that beyond a certain point, increasing parameters and epochs no longer seems to

dim_hl_1	dim_hl_2	num_epochs	total_parameters	test_loss	test_accuracy
128	128	25	118282	0.150310	0.9789
128	64	15	109386	0.131181	0.9757
64	64	20	55050	0.135872	0.9734
64	32	15	52650	0.111240	0.9733
32	32	10	26506	0.127495	0.9660

Table 1: Results of diferents configurations of feedforward webs in MNIST

improve the model. Therefore, I suggest starting with simpler configurations and gradually increasing the model's complexity until we find the sweet spot—the threshold between fewer parameters and high accuracy $\square$

Problem Sub-task 2

Choose one aspect or issue you would like to understand better, research it, and write about 1 page summarizing your findings.

- What is Keras? What is Jax? What is NumPy? How are they related?

answer. To deepen my understanding of these technologies and contrast their capabilities, I went straight to the source: the official Keras website (keras.io) and the technical documentation for JAX hosted by Google (jax.readthedocs.io). Below, I detail what I discovered during my investigation, followed by a comparative analysis and an explanation of how these two powerful tools intersect in the modern machine learning landscape.

Findings from the Official Research Keras: The Human-Centric Interface During my exploration of the official Keras portal, I discovered that this ecosystem defines itself as a deep learning API "designed for human beings," with its core philosophy centered on reducing cognitive load for the developer. The most crucial takeaway from their current documentation is its evolution: Keras is no longer just a rigid sub-module tied exclusively to TensorFlow.

With the release of Keras 3, it has transformed into a multi-backend framework. I found that Keras now acts as a unified, high-level abstraction layer that allows the exact same neural network code to be executed on top of three different engines: TensorFlow, JAX, or PyTorch. Its primary utility is to shield users from the complexity of manipulating low-level tensors, manual gradients, and intricate hardware configurations, empowering researchers to focus on conceptual model architecture through modular and highly readable components.

JAX: High-Performance Numerical Computing Conversely, when digging into the official JAX documentation, I encountered a radically different design philosophy. JAX does not pitch itself as a deep learning framework out of the box; instead, it is described as a library for scientific computing and program transformations. Officially, JAX is the marriage of Autograd (for automatic differentiation) and XLA (Accelerated Linear Algebra, a compiler optimized for mathematical operations).

I verified that JAX implements a version of the NumPy API (jax.numpy) that runs natively on hardware accelerators like CPUs, GPUs, and TPUs. Built on the principles of

functional programming, JAX offers powerful composable transformations such as `jit` (just-in-time compilation), `grad` (automatic differentiation of any order), and `vmap` (automatic vectorization), giving developers total mathematical control and extreme execution speeds.

Comparative Analysis By analyzing the official documentation of both tools, it becomes clear that they operate at different levels of the development stack and serve distinct structural purposes:

Level of Abstraction: Keras sits at the top of the pyramid, providing high-level building blocks like layers (Layers), models (Models), optimizers, and metrics. JAX sits at the foundational layer of scientific computing; it does not inherently understand "neural networks," but rather provides the raw mathematical primitives required to build them from scratch.

Programming Paradigm: Keras primarily embraces an object-oriented approach where layers maintain internal states (such as weights). JAX strictly adheres to functional programming using pure functions, meaning states and parameters must be managed explicitly outside of the transformed functions.

Core Objective: Keras prioritizes rapid prototyping, developer productivity, and seamless deployment. JAX prioritizes research flexibility, raw mathematical optimization, and maximum hardware acceleration for complex algorithms.

The Relationship Between Keras and JAX Despite their contrasting philosophies—one focusing on human ergonomics and the other on raw mathematical performance—the relationship between Keras and JAX is deeply symbiotic. They are not rival technologies competing for dominance; rather, they complement each other to create an optimized workflow.

This technological convergence is realized because JAX can now act as the official computational engine (backend) for Keras. When I configure Keras to run on top of JAX, I get the best of both worlds: I can design, train, and evaluate complex neural networks using the clean, intuitive syntax of Keras, while under the hood, that code is automatically compiled and optimized via JAX and XLA.

Ultimately, JAX provides the high-powered engine and hardware control, while Keras provides the ergonomic chassis and steering wheel, allowing developers to drive that computational power with absolute efficiency. $\square$